

Evolution strategies

Evolutionary robotics

Nicolas Bredeche

Sorbonne Université
ISIR, UMR 7222
Paris, France
nicolas.bredeche@sorbonne-universite.fr



UE Robotique et Apprentissage
M1 informatique, parcours Androïde

rev. 2022-02-25

Outline

2

- Part 1 : evolution strategies
 - ▶ Concepts and definitions
 - ▶ Population-based methods
 - ▶ Probability-weighted averaging methods
- Part 2 : evolutionary robotics
 - ▶ Concepts and definition
 - ▶ ES as reinforcement learning method: direct policy search

Evolution Strategies

Elements

4

problem: $x^* = \operatorname{argopt}_{x \in X} f(x)$

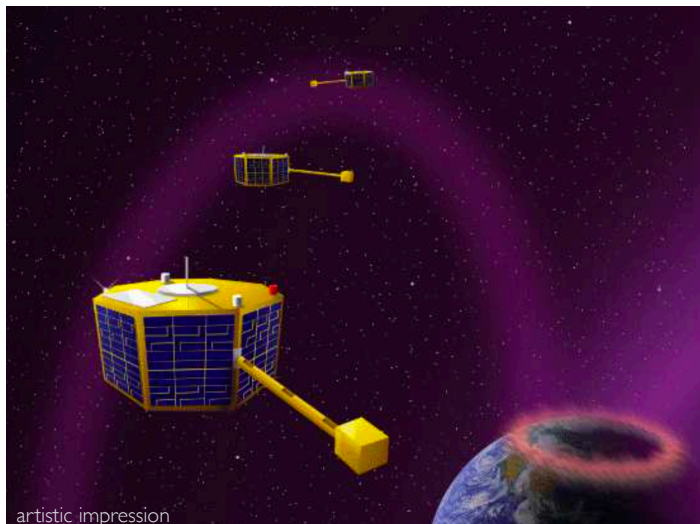
candidate solution: $a := (x, f(x))$

$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

- **black-box stochastic optimisation** algorithms
 - derivative-free
 - population-based
 - iterative
 - stochastic operators
 - versatile wrt. search space (GA, GP, EP, **ES**, etc.)

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

- Elements
 - candidate solutions — $x \in X$
 - objective function — f
 - performance value, or “fitness” — $f(x)$
 - objective: maximise (or minimise) the fitness — note: $\max(F) = \min(-F)$



$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

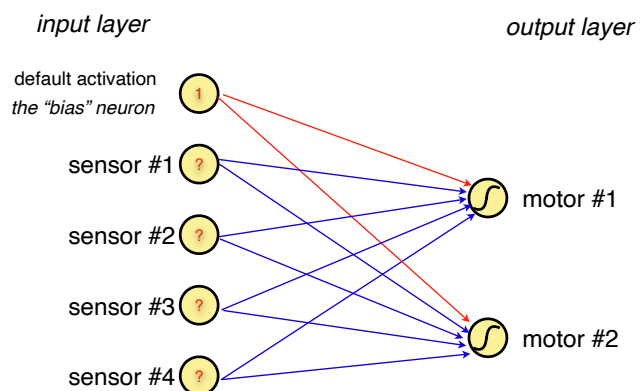
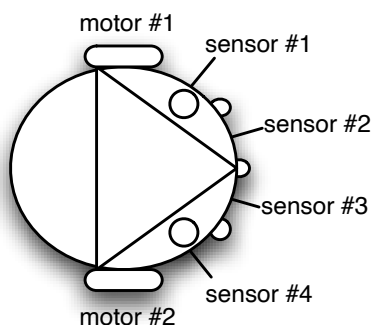
with x , the spatial configuration

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

the best antenna is obtained w/ parameters x^*

[Horny et al., 2006, NASA] "Automated Antenna Design with Evolutionary Algorithms"

Blackbox optimisation and robotics



NN weights as parameters in $\mathbb{R}^{10}$

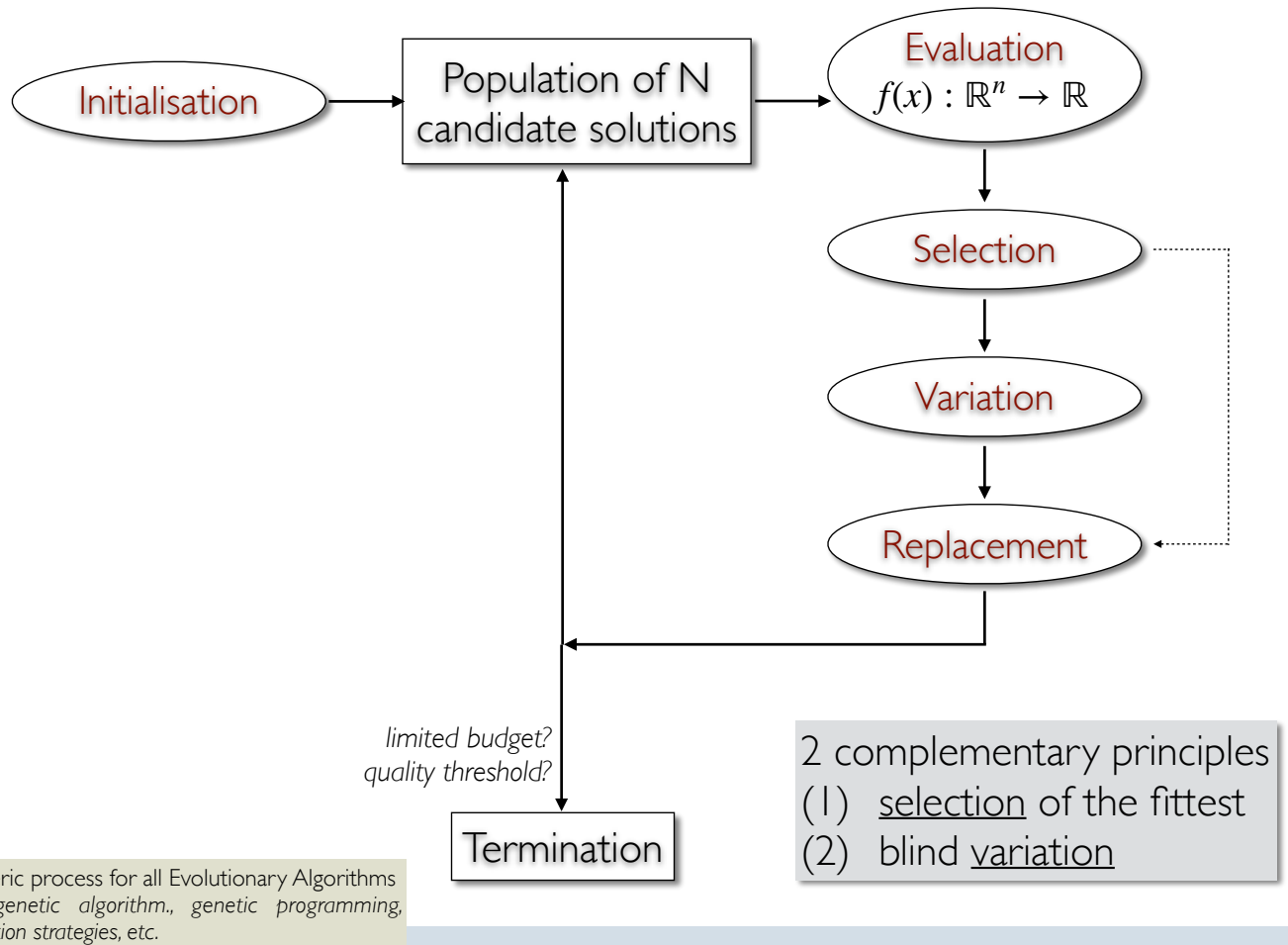
$$\text{output}_j = \tanh\left(\sum_i w_{i,j} * \text{input}_i\right)$$

$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

with x , the neural network weights

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

the best trajectory is obtained w/ parameters x^*



- Search space is in $\mathbb{R}^n$
- (μ, λ) notation
 - μ : number of parents ; λ : number of offspring
 - $(\mu + \lambda)$: selection in parents U offsprings
 - (μ, λ) : selection in offsprings only

Evolution Strategies

(1+1)-ES algorithm

Naive (1+1)-ES algorithm

12

init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

for i in range(evaluations):

$x' = x + \sigma N(0, I)$

 if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

Details in the next slide...

a candidate solution
(the initial "parent")

number of dimensions
(depends on the problem)

init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

mutation step size

for i in range(evaluations):

$x' = x + \sigma N(0, I)$

if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

gaussian mutation

Example of implementation:

(...)

for d in dimensions:

$x'[d] = x[d] + \text{rand.gaussian}(\text{sigma})$

(...)

the "child"

the "parent"

here, we assume this is
a maximisation problem

x : candidate solution, vector of n real values

σ : mutation step-size

$N(0, I)$: multivariate normal distribution with mean=0 and unity covariance matrix

Naive (1+1)-ES algorithm

init: $x \in \mathbb{R}^n$

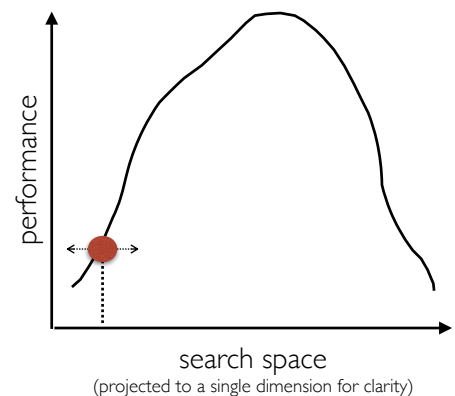
init: $\sigma > 0$

for i in range(evaluations):

$x' = x + \sigma N(0, I)$

if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

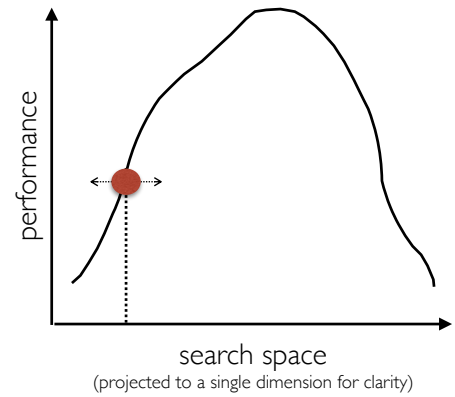


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```



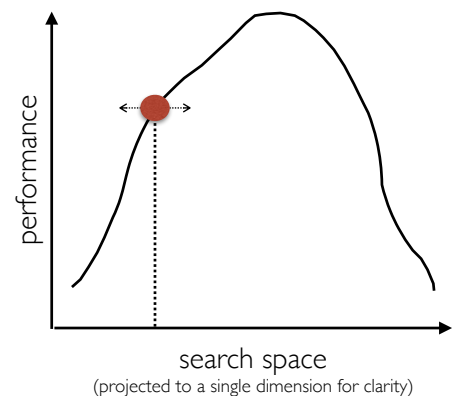
```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

sigma is too small!!



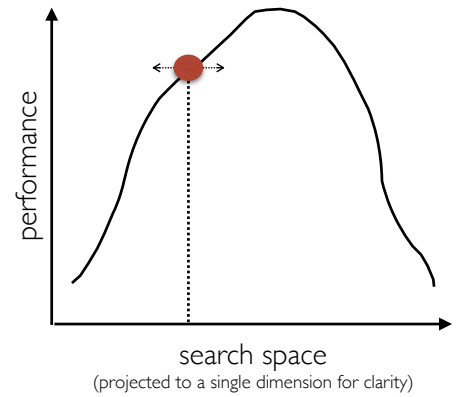
```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if  $\text{fitness}(x') \geq \text{fitness}(x)$ :
         $x = x'$ 

```

sigma is too small!!



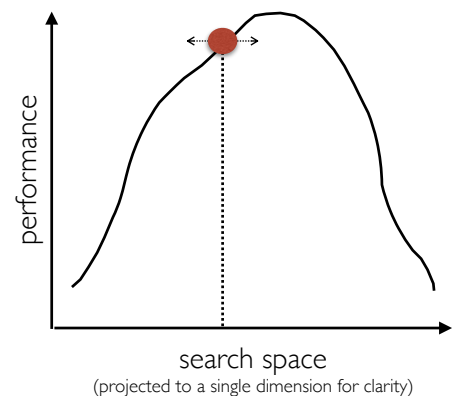
```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if  $\text{fitness}(x') \geq \text{fitness}(x)$ :
         $x = x'$ 

```

sigma is too small!!



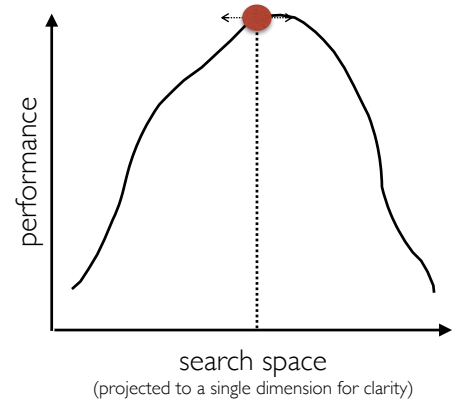
```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

sigma is too small!!

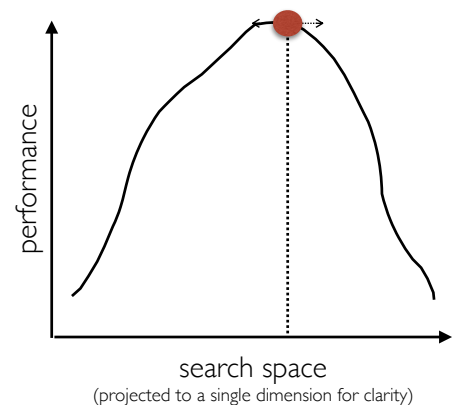


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

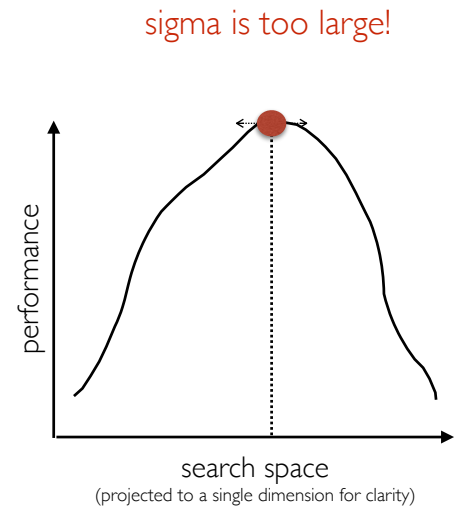


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

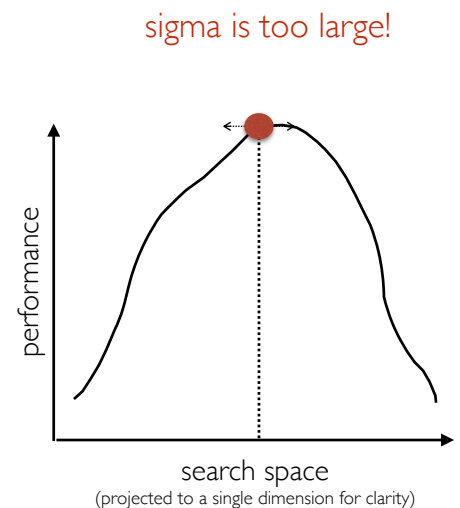


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

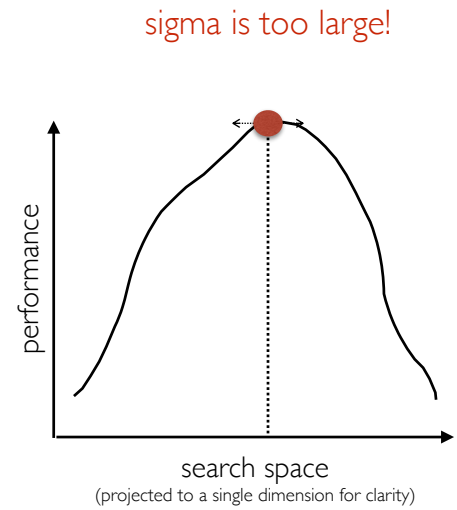


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range( evaluations ):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```



oscillates around the optimum

Naive (1+1)-ES algorithm, 1/5th rule

```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range( evaluations ):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```

}

the “one fifth” rule

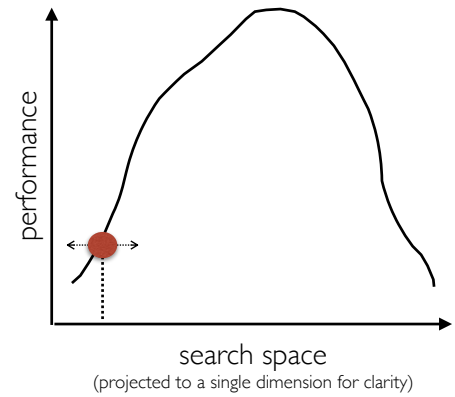
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range( evaluations ):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



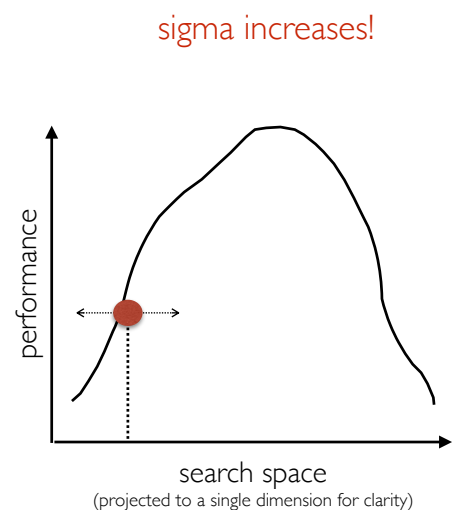
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range( evaluations ):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



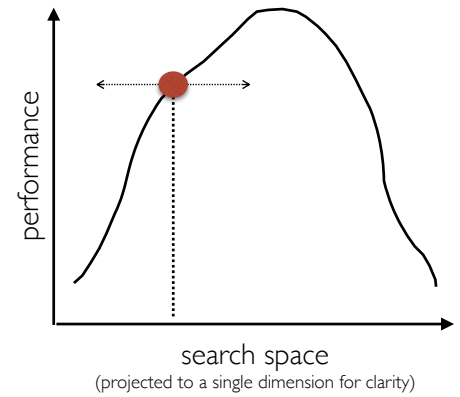
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



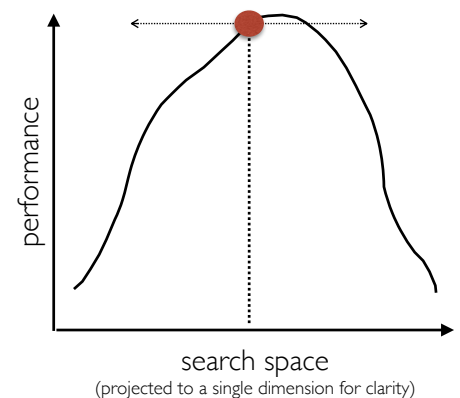
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



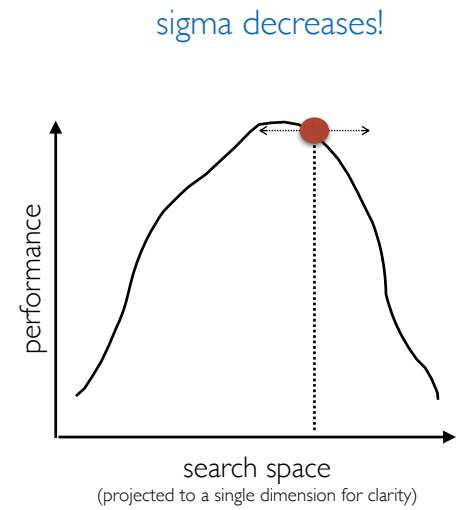
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range( evaluations ):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



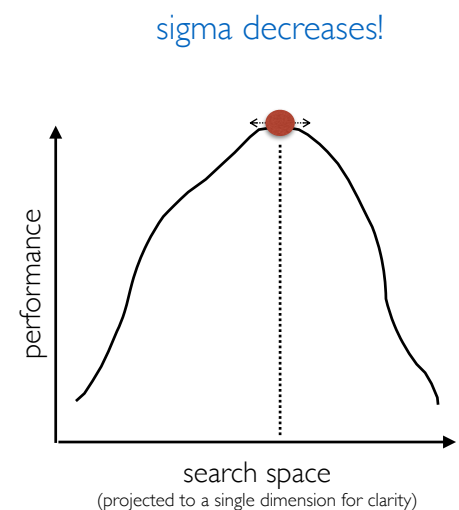
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range( evaluations ):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



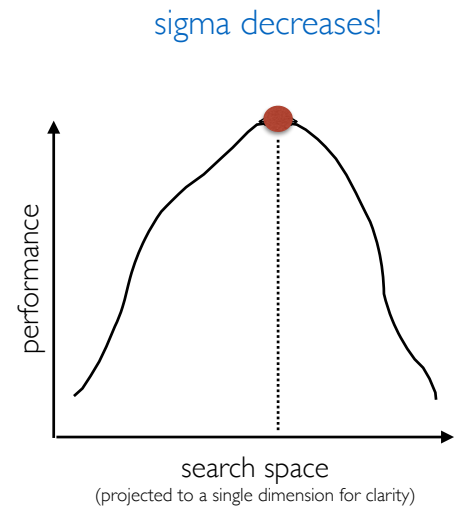
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range( evaluations ):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



adaptative convergence

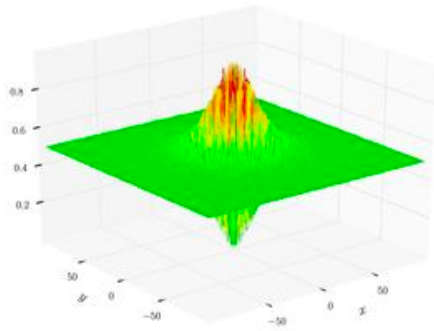
- Algorithme (1+1)-ES, règle des 1/5e
 - convergence adaptative
 - limité aux fonctions convexes
 - peu robuste au bruit —> faire plusieurs évaluations!

Evolution Strategies

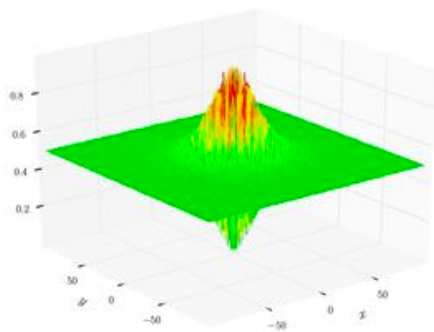
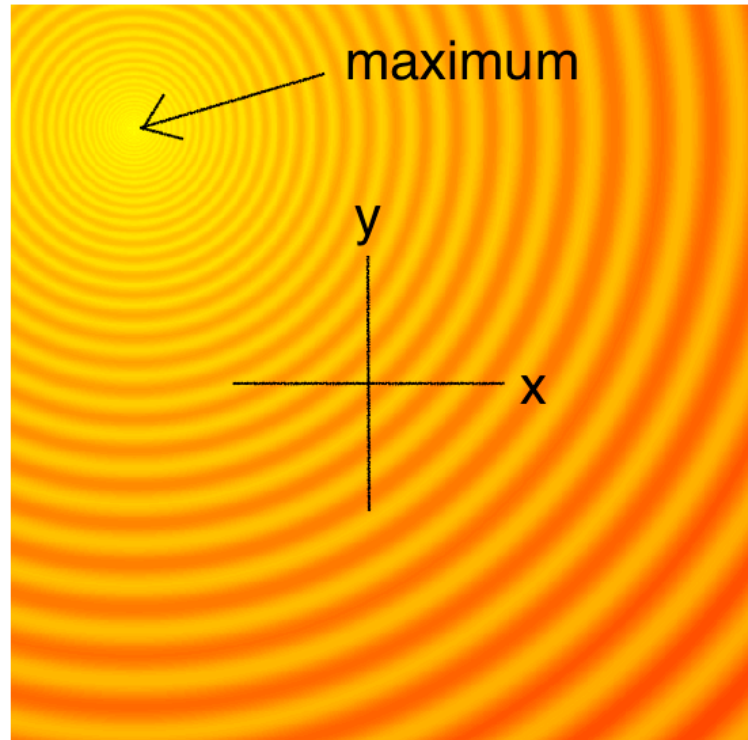
$(\mu=1, \lambda=n)$ -ES algorithm

34

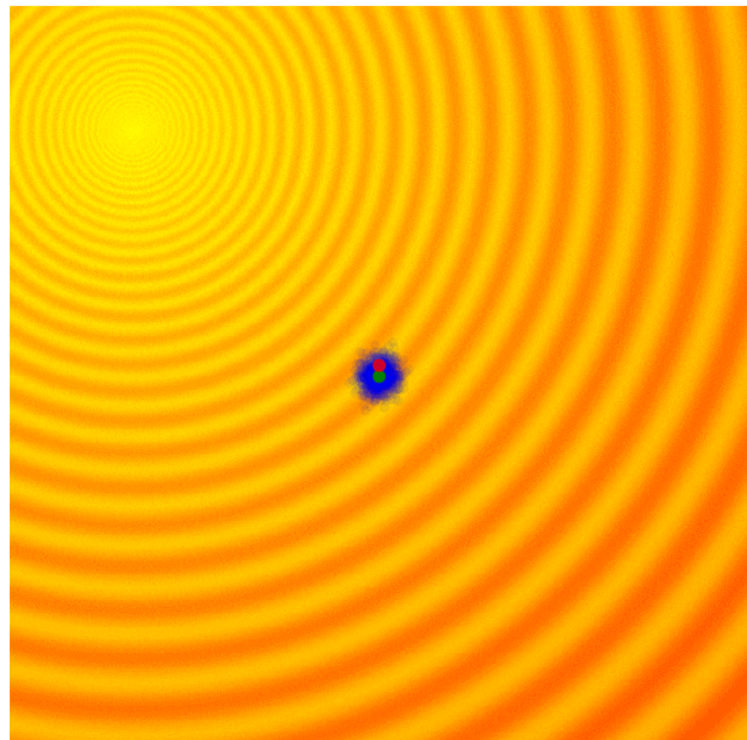
- Algorithme $(1,n)$ -ES
 - génère n individus en appliquant un bruit gaussien
 - sélectionne le *meilleur* individu
 - recommence

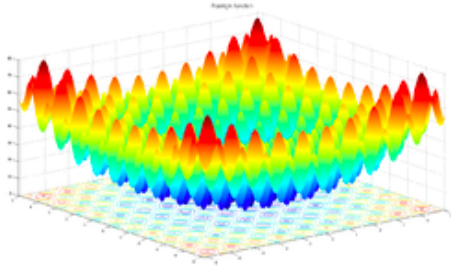


$$f(x, y) = 0.5 + \frac{\sin^2(x^2 - y^2) - 0.5}{[1 + 0.001(x^2 + y^2)]^2}$$



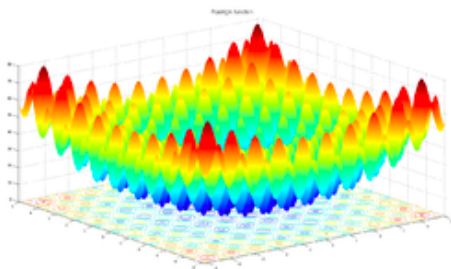
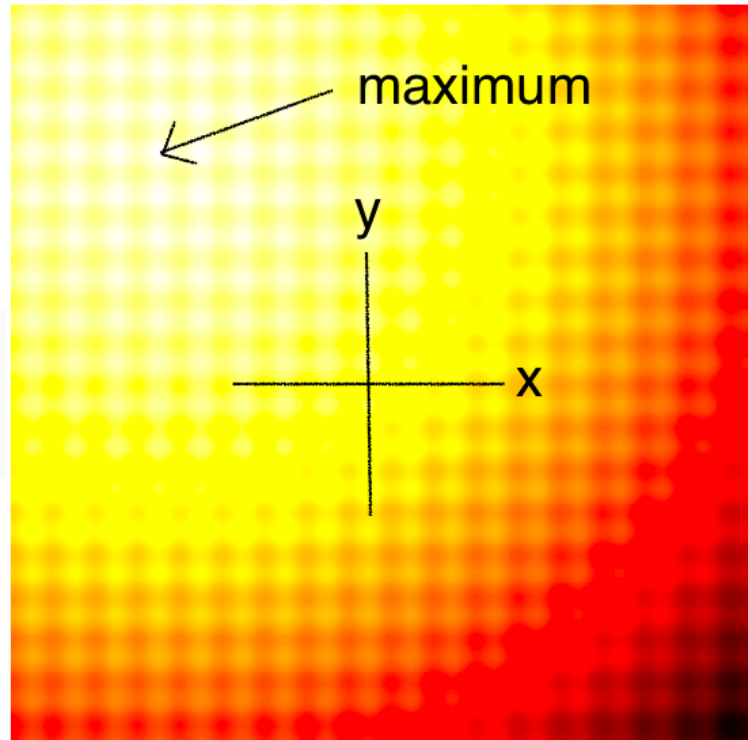
$$f(x, y) = 0.5 + \frac{\sin^2(x^2 - y^2) - 0.5}{[1 + 0.001(x^2 + y^2)]^2}$$





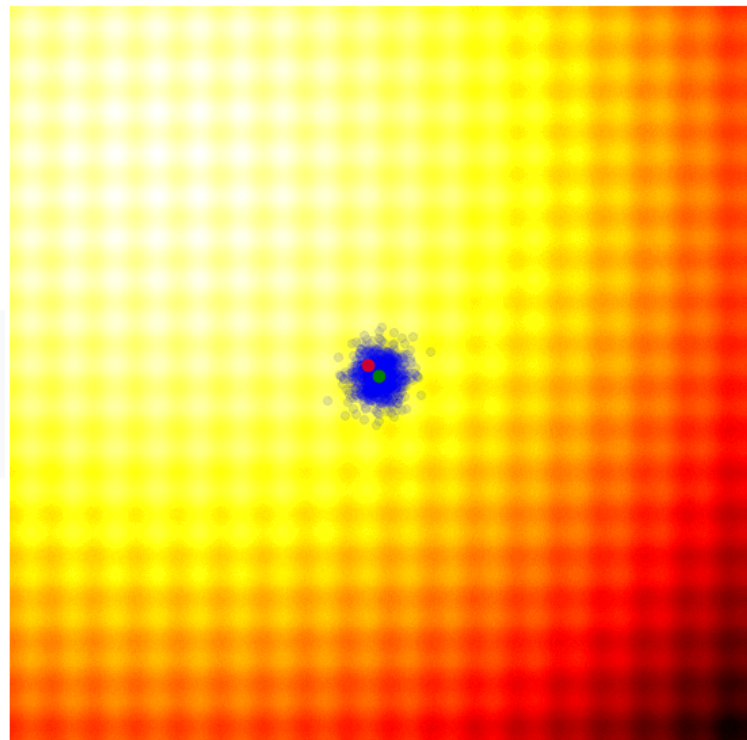
$$f(\mathbf{x}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

where: $A = 10$



$$f(\mathbf{x}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

where: $A = 10$



- Algorithm (1,n)-ES
 - avoid local optima ...to some extent
 - slow convergence, fixed step-size
 - exploit only the single best individual... natural extension $\mu > 1$

Evolution Strategies

(μ, λ) -ES and $(\mu + \lambda)$ -ES algorithms

- Algorithm
 - select the best μ individuals (parents)
 - create λ individuals (offspring)
 - ▶ to build a new individual out of λ :
 - take a random individual among the μ parents
 - apply a mutation to get the new child (e.g. gaussian mutation)
 - build the new population
 - ▶ (μ, λ) -ES method: the new population contains only λ individuals
 - ▶ $(\mu + \lambda)$ -ES method: the new population contains both $\mu + \lambda$ individuals
 - continue

With $\mu < \lambda$

suggested: initial population size = λ

Evolution Strategies

Probability-weighted averaging method

$$(1+1) \text{ ES} \quad \begin{cases} x = m + \sigma \mathcal{N}(0, I) \\ \text{if}(f(x) > f(m)) : m \leftarrow x \end{cases}$$

$$(\mu/\mu, \lambda) \text{ ES} \quad \begin{cases} x_i = m + \sigma \mathcal{N}_i(0, I) \\ m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda} \end{cases}$$

μ/μ : recombination of all parents
 → provides a **single parental centroid**

Cross-Entropy Method (CEM)

44

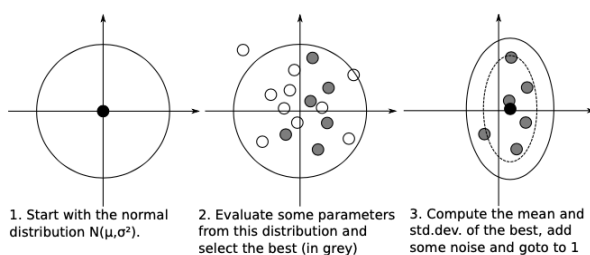


Fig. 2. Schematic view of the Cross-Entropy method.

θ_i : parameter set of solution i (previously: " x ")
 J_i : fitness of solution i (previously: " $f(x)$ ")

Algorithm 1 CEM_iter

Require: $\{(\theta_i, J_i)\}_{i=0 \dots N}$: set of N solution-value pairs
 ρ : proportion of the best solutions to use for the update
 σ_{noise}^2 : additional noise term

Sort the θ_i according to J_i

Compute the set S_ρ of the $\max(1, N \times \rho)$ best solutions

$\mu \leftarrow \text{mean}(S_\rho)$

$\sigma^2 \leftarrow \text{std.dev}(S_\rho) + \sigma_{noise}^2$

return mean μ and std.dev. σ^2

set containing the best and $N \times \rho$ others

● Probability-weighted averaging method

► Elements

- μ : mean of distribution
- σ^2 : standard deviation

► Principles

- update μ and σ^2 through sampling, with selection pressure ρ

- Cross-Entropy Method
 - exploite l'ensemble de la population
 - convergence adaptative
 - évite les optima locaux
 - exploration uniforme du voisinage, et à pas fixe

Evolution Strategies

CMA-ES algorithm

$$(\mu/\mu, \lambda) \text{ ES} \quad \left\{ \begin{array}{l} x_i = m + \sigma \mathcal{N}_i(0, \mathbf{C}) \\ m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda} \end{array} \right.$$

$\mathbf{C}$: covariance matrix, $\mathbf{C} \in \mathbb{R}^{n \times n}$

CMA-ES

48

- Co-variance Matrix Adaptation Evolution Strategy

- ▶ self-adaptive search, (nearly) parameter-free
- ▶ invariance to
 - order-preserving transformations (only depends on rank)
 - scale (if initial step-size σ and initial search point chosen accordingly)
 - scaling of variable (diagonal invariance)
 - angle preserving transformations (rotation, reflection, translation)
 - invertible linear transformation
- ▶ works in large dimension (100+)

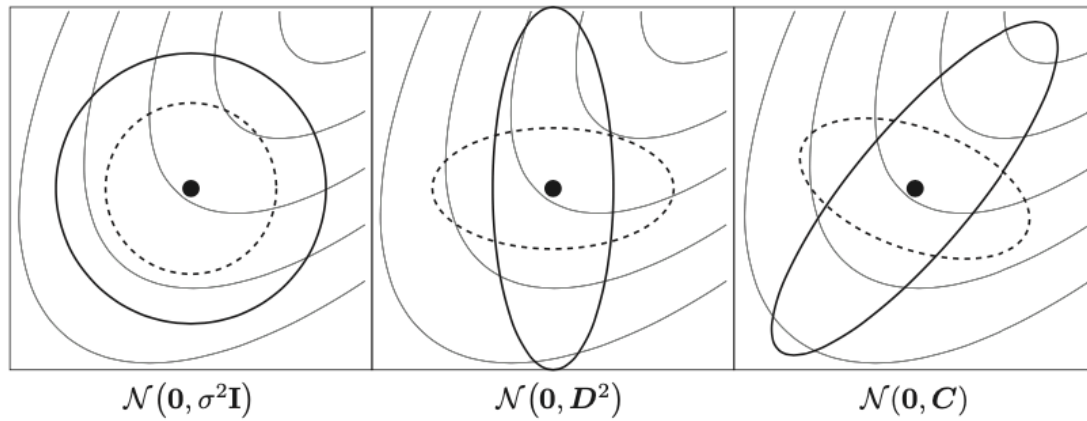
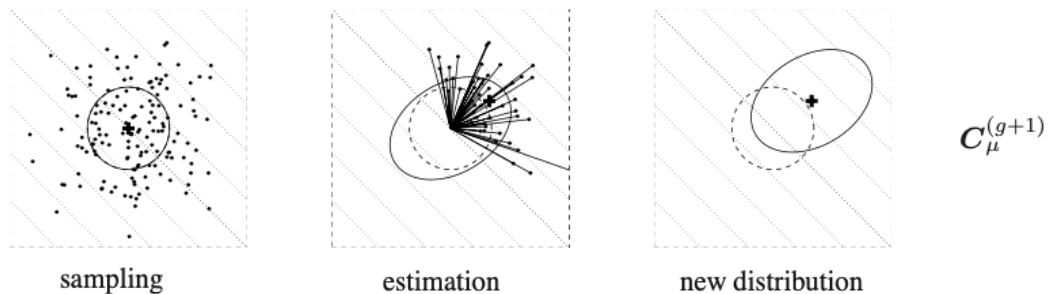


Figure 1: Ellipsoids depicting one- σ lines of equal density of six different normal distributions, where $\sigma \in \mathbb{R}_{>0}$, $\mathbf{D}$ is a diagonal matrix, and $\mathbf{C}$ is a positive definite full covariance matrix. Thin lines depict possible objective function contour lines

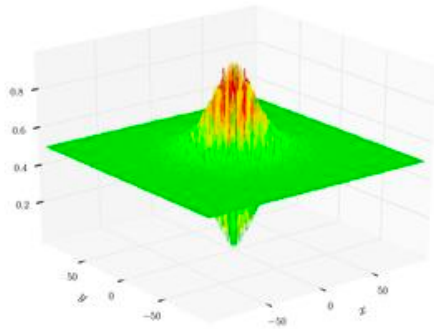
Hansen (2016) The CMA Evolution Strategy: a Tutorial. Arxiv.

Covariance matrix (cont.)

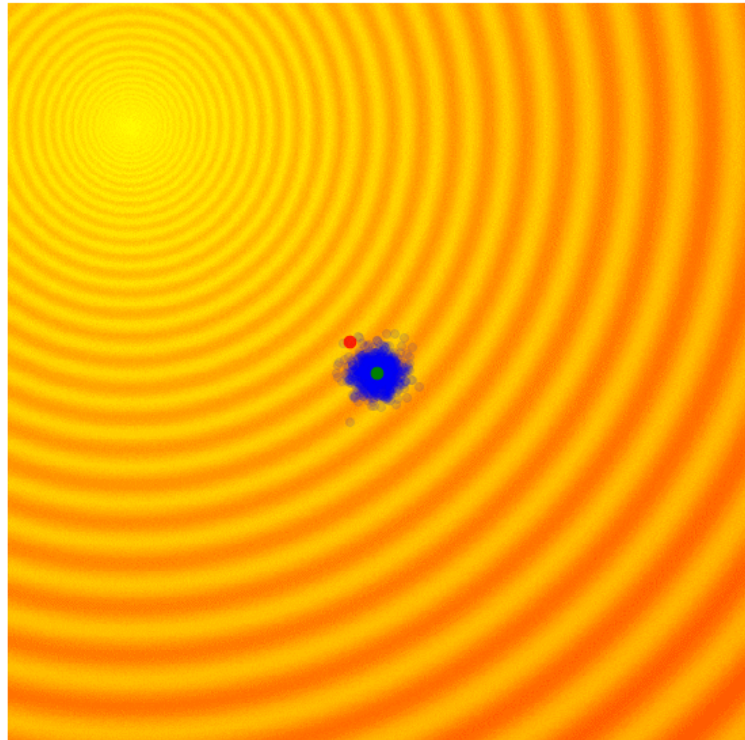
50



Hansen (2016) The CMA Evolution Strategy: a Tutorial. Arxiv.



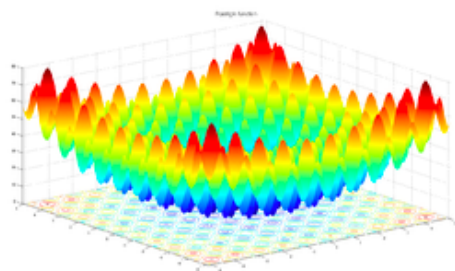
$$f(x, y) = 0.5 + \frac{\sin^2(x^2 - y^2) - 0.5}{[1 + 0.001(x^2 + y^2)]^2}$$



wikipedia

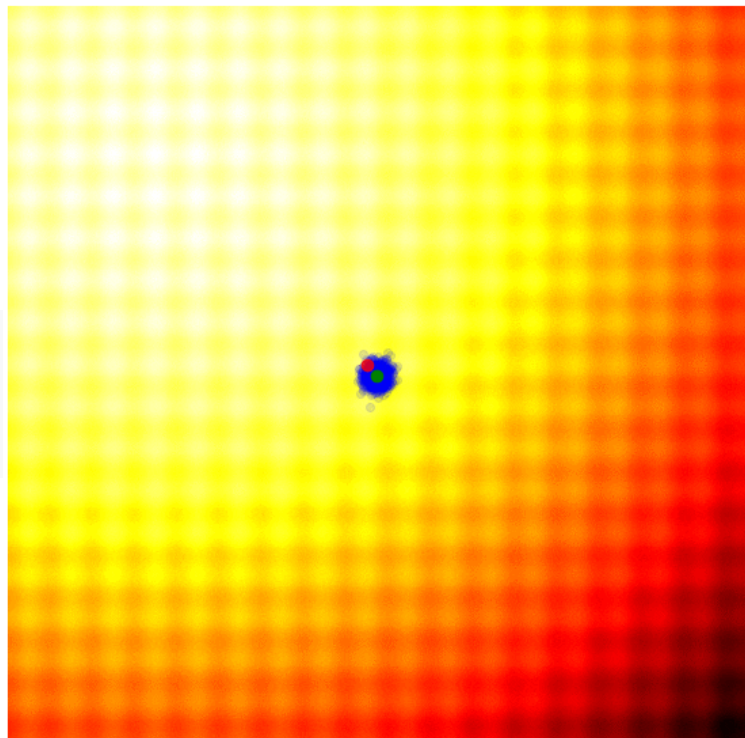
Ha, D. (2017) A Visual Guide to Evolution Strategies, blog.otoro.net, 2017

Rastrigin-2D function



$$f(\mathbf{x}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

where: $A = 10$



wikipedia

Ha, D. (2017) A Visual Guide to Evolution Strategies, blog.otoro.net, 2017

Input: $\mathbf{m} \in \mathbb{R}^n$, $\sigma \in \mathbb{R}_+$, λ

Initialize: $\mathbf{C} = \mathbf{I}$, and $\mathbf{p}_c = \mathbf{0}$, $\mathbf{p}_\sigma = \mathbf{0}$,

Set: $c_c \approx 4/n$, $c_\sigma \approx 4/n$, $c_1 \approx 2/n^2$, $c_\mu \approx \mu_w/n^2$, $c_1 + c_\mu \leq 1$, $d_\sigma \approx 1 + \sqrt{\frac{\mu_w}{n}}$,
and $w_{i=1 \dots \lambda}$ such that $\mu_w = \frac{1}{\sum_{i=1}^{\mu} w_i^2} \approx 0.3 \lambda$

While not terminate

$\mathbf{x}_i = \mathbf{m} + \sigma \mathbf{y}_i$, $\mathbf{y}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$, for $i = 1, \dots, \lambda$ sampling

$\mathbf{m} \leftarrow \sum_{i=1}^{\mu} w_i \mathbf{x}_{i:\lambda} = \mathbf{m} + \sigma \mathbf{y}_w$ where $\mathbf{y}_w = \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda}$ update mean

$\mathbf{p}_c \leftarrow (1 - c_c) \mathbf{p}_c + \mathbb{1}_{\{\|\mathbf{p}_\sigma\| < 1.5\sqrt{n}\}} \sqrt{1 - (1 - c_c)^2} \sqrt{\mu_w} \mathbf{y}_w$ cumulation for $\mathbf{C}$

$\mathbf{p}_\sigma \leftarrow (1 - c_\sigma) \mathbf{p}_\sigma + \sqrt{1 - (1 - c_\sigma)^2} \sqrt{\mu_w} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_w$ cumulation for σ

$\mathbf{C} \leftarrow (1 - c_1 - c_\mu) \mathbf{C} + c_1 \mathbf{p}_c \mathbf{p}_c^T + c_\mu \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^T$ update $\mathbf{C}$

$\sigma \leftarrow \sigma \times \exp\left(\frac{c_\sigma}{d_\sigma} \left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|} - 1\right)\right)$ update of σ

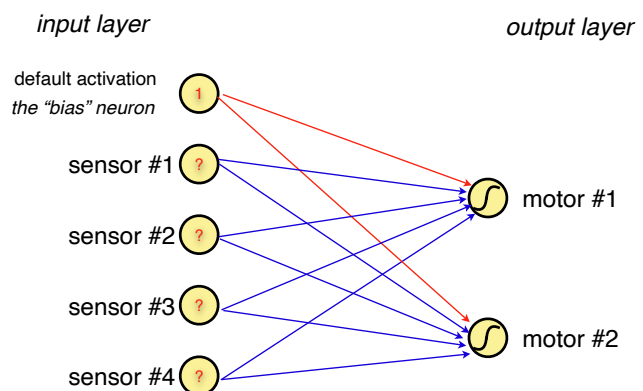
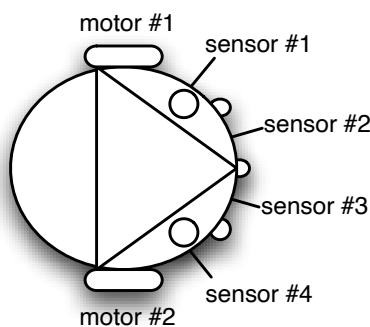
Hansen (2016) The CMA Evolution Strategy: a Tutorial. Arxiv.

- CMA-ES
 - exploite l'ensemble de la population
 - convergence adaptative
 - exploration adaptative du voisinage
 - évite les optima locaux

Evolutionary Robotics

Concepts and definitions

Blackbox optimisation and robotics



NN weights as parameters in $\mathbb{R}^{10}$

$$output_j = \tanh\left(\sum_i w_{i,j} * input_i\right)$$

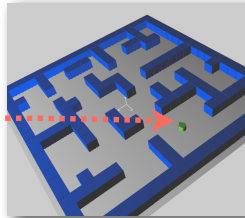
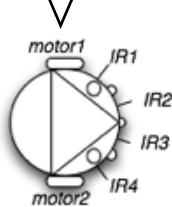
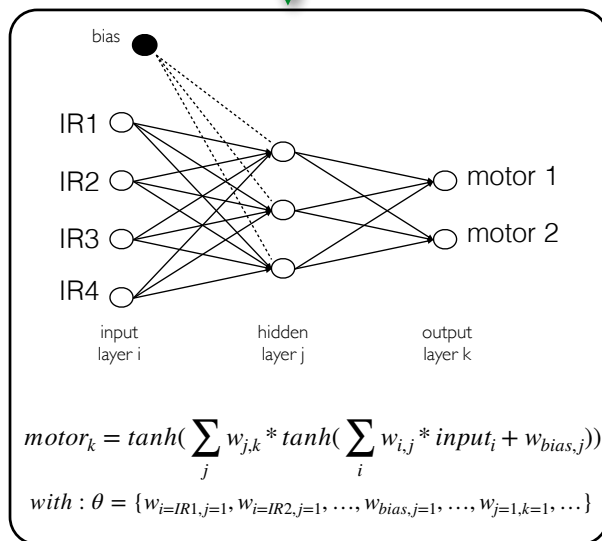
$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

with x , the neural network weights

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

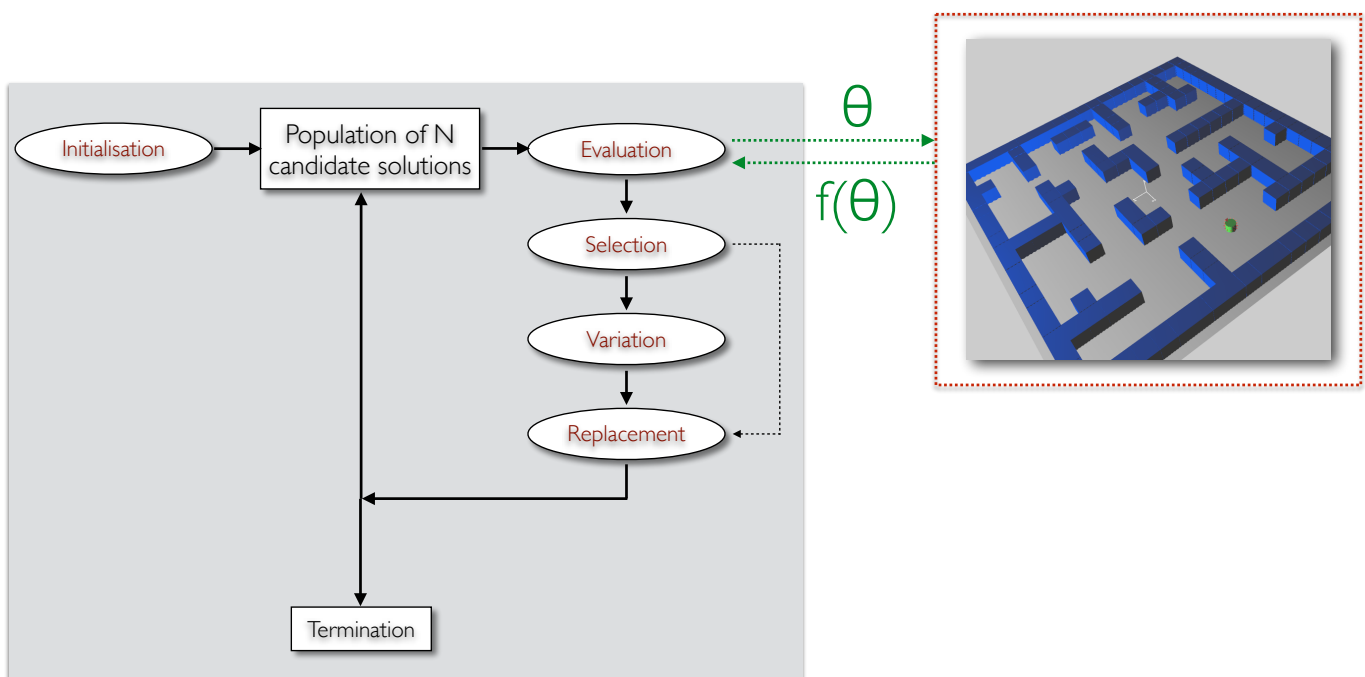
the best trajectory is obtained w/ parameters x^*

neural net weights θ



fitness
 $f(\theta)$

Evolutionary Robotics



θ : parameters

f : objective function

$f(\theta)$: fitness

$$\Pi_{\theta} : S \rightarrow A$$

$$J(\theta) : \mathbb{R}^n \rightarrow \mathbb{R}$$

$$\theta^* = \operatorname{argopt}_{\theta \in \Theta} J(\theta)$$

- ES as a reinforcement learning method

- ▶ Direct **policy** search: find the best θ w.r.t. the objective function
- ▶ *Viewpoint from ES: just another blackbox optimisation problem*

θ : parameters ($\Rightarrow x$)

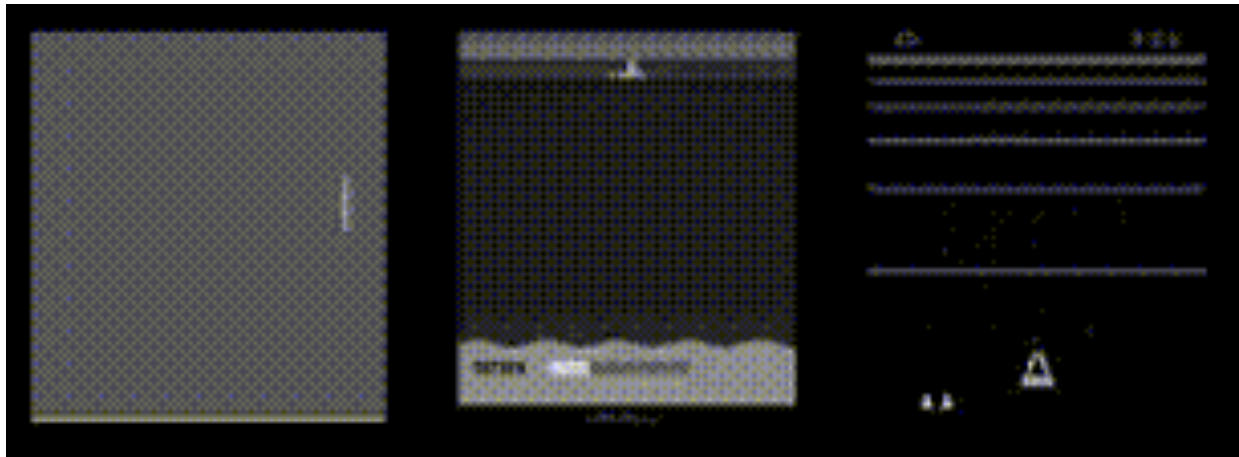
J : performance function

$J(\theta)$: "return" (sum of immediate rewards)



Evolutionary Robotics

ES for reinforcement learning



- 51 Atari games [Brockman et al., 2016]

Salimans et al. (2017) — OpenAI

Convolutional Neural Network for ATARI 2600 games

62

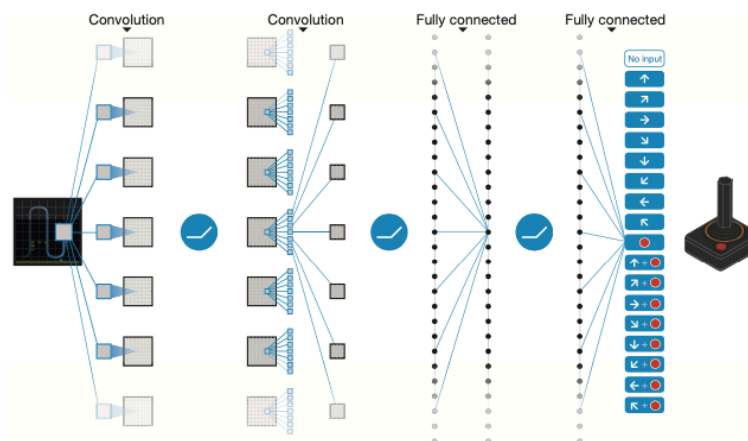


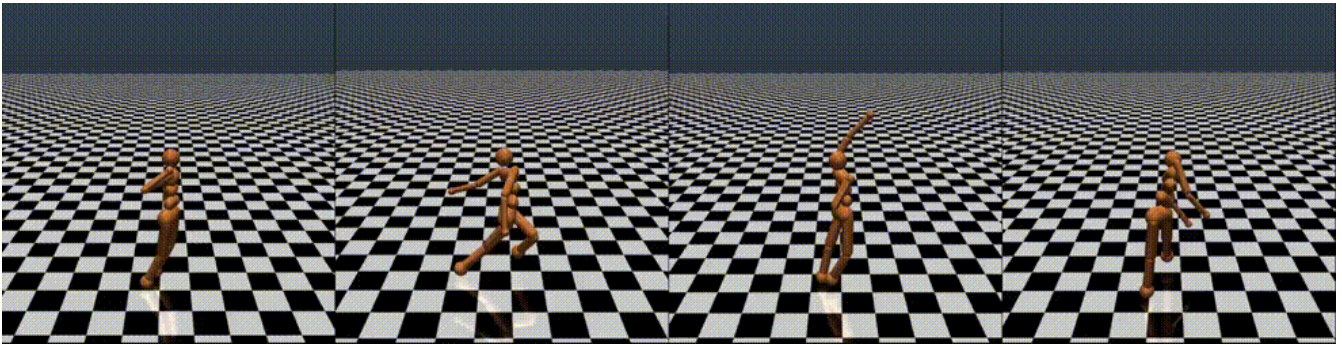
Figure 1 | Schematic illustration of the convolutional neural network. The details of the architecture are explained in the Methods. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ , followed by three convolutional layers (note: snaking blue line

symbolizes sliding of each filter across input image) and two fully connected layers with a single output for each valid action. Each hidden layer is followed by a rectifier nonlinearity (that is, $\max(0, x)$).

→ ~50 millions NN weights, 2 millions are learned

- Control: A3C, 320 million frames, ~1 day [Mnih et al., 2016]
- Budget per game for ES: 1 billion frames
 - ▶ ES does not compute backprop nor value function => same time
 - ▶ ES easier to parallelise: swap fitness, not gradient => ~1 hour
- Result: ES better on 23 games, worse on 28 games

Mnih et al. (2016) — Google Deepmind



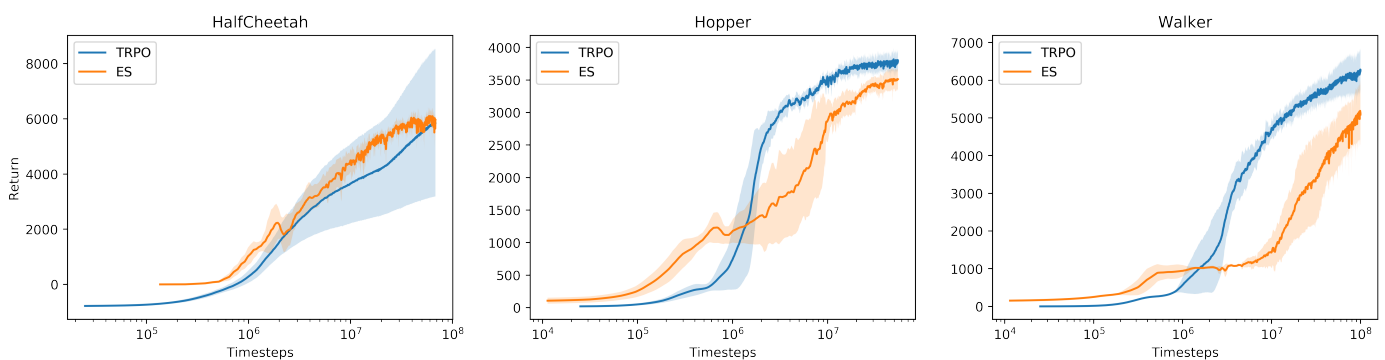
- multilayer perceptrons
 - ▶ 2x 64-unit hidden layers
 - ▶ hyperbolic tan nonlinearities

Also tested with *HalfCheetah*, *Hopper*, *InvertedDoublePendulum*, *InvertedPendulum* and *Swimmer*

Salimans et al. (2017) — OpenAI

ES vs. gradient-based RL

64



- Experiments
 - Similar performance after 5 million time steps
 - ES requires more samples (3x to 10x)*

*
10x more samples in “hard” environments (*Walker3D*, *Hopper*)
3x more samples in “simple” environments (*HalfCheetah*, *InvertedDoublePendulum*, *InvertedPendulum* and *Swimmer*)

Salimans et al. (2017) — OpenAI

- Comparing ES and state-of-the-art RL methods
 - ▶ ES is competitive with state-of-the-art RL methods
 - ▶ ES explores more diverse solutions
 - ▶ ES explores more robust solutions
 - ▶ ES requires much more samples (3x, 10x) than RL methods
 - ▶ But...
 - ES does not compute back-propagation (3x time gain)
 - ES benefits from easy parallelisation (only fitness values exchanged not full gradient info)

Further reading: <https://eng.uber.com/deep-neuroevolution/>

Conclusions

- Evolutionary Strategies...
 - as a blackbox optimisation method
 - ▶ performs derivative-free optimisation
 - ▶ addresses non-convex problem
 - as a policy search method for reinforcement learning
 - ▶ addresses large continuous domain (states and actions)
 - ▶ handles credit assignment over long time-scale
 - ...and can be implemented in a highly parallel fashion